Universidad Mayor
Facultad de Ciencias, Ingeniería y Tecnología
Escuela de Ingeniería Civil
Arquitectura de Desarrollo (Móvil y Web)
INGT1003

UNIV. MAYOR

DOCUMENTO DE PROPUESTA

SeñaCL

Traductor de Lengua de Señas Chilena

Integrantes:

Concha Javier Ignacio

Urbina Garcidueñas Luis Jesús

Vega Graterol Juan Carlos

Docente: Muñoz Soto Carlos Mauricio

Santiago de Chile, Chile

20 / 09 / 2026

HITO 1

CONTENIDOS

Índice

— **Introducción** 1

— **Desarrollo** 2

Propuesta de Valor 2

Arquitectura de Información 2

Arquitectura Cliente-Servidor y Capas 3

Selección de Tecnologías y Decisiones de Arquitectura 6

— **Coste Monetario del Proyecto** 7

— **Conclusión** 8

— **Bibliografía** 9

— **Anexos** 10

FIGURAS

Figura 1 — Mapa del producto 3

Figura 2 — Diagrama de arquitectura 5

SECCIÓN 1

Introducción

La comunicación efectiva es un pilar fundamental para la integración social, el desarrollo personal y la equidad. Sin embargo, en la actualidad persiste una profunda barrera comunicativa entre la comunidad con discapacidad auditiva y las personas oyentes que desconocen la lengua de señas. En Chile, según los datos proyectados por el censo de 2024 respecto a la población con discapacidad, aproximadamente dos millones de personas presentan problemas de audición y/o dificultades para comunicarse (Instituto Nacional de Estadísticas, 2026). Esta brecha lingüística excluye a una parte significativa de la población del acceso autónomo y fluido a servicios básicos, educación, salud y oportunidades laborales, haciendo imperativa la creación de herramientas tecnológicas e inclusivas que actúen como puentes de interacción en la vida cotidiana.

Para mitigar esta problemática, este proyecto plantea el desarrollo de una aplicación móvil basada en visión por computadora e inteligencia artificial, diseñada para funcionar como un traductor unilateral de Lengua de Señas Chilena (LSCh). La herramienta permitirá a los usuarios utilizar la cámara de su dispositivo móvil para enfocar a una persona comunicándose en LSCh y procesar los gestos en tiempo real, generando una traducción automática y simultánea en formatos de texto y audio.

El objetivo principal de este documento es presentar la propuesta formal, los alcances y la viabilidad técnica del proyecto del traductor de LSCh. A través de este reporte, se busca establecer las bases tecnológicas y metodológicas que guiarán la construcción de la aplicación, definiendo claramente el problema a resolver y las herramientas de software necesarias para el procesamiento del lenguaje y la visión artificial.

A lo largo de las siguientes páginas, el lector encontrará un desglose técnico y estructurado de la iniciativa. En la sección de **Desarrollo**, se detallará la arquitectura del sistema, el flujo de funcionamiento de la aplicación móvil y las tecnologías involucradas. Posteriormente, en las **Conclusiones**, se expondrán los

hitos logrados en esta fase de propuesta. Finalmente, el documento incluye la **Bibliografía** que sustenta la investigación y los **Anexos** con material de respaldo.

SECCIÓN 2

Desarrollo

Propuesta de Valor

> Para las personas con discapacidad auditiva y los oyentes que enfrentan la barrera del desconocimiento de la lengua de señas en interacciones cotidianas, nuestro producto entrega una **traducción automática, unidireccional y en tiempo real de la Lengua de Señas Chilena (LSCh) a texto y audio**.

Arquitectura de Información

La aplicación móvil está diseñada para minimizar la fricción en la comunicación. La estructura jerárquica principal consta de:

1. **Módulo de Traducción (Pantalla Principal):** Interfaz limpia de acceso a la cámara para la captura de señas, visualizador de texto en tiempo real y emisión de audio.
2. **Historial de Sesión:** Registro temporal de las frases traducidas almacenadas en la base de datos para recuperar el contexto de la conversación.
3. **Configuración:** Ajustes de accesibilidad (velocidad de síntesis de voz, tamaño de texto).

FIGURA 1 — MAPA DEL PRODUCTO



Figura 1 Mapa del producto

Arquitectura Cliente-Servidor y Capas

Para optimizar el rendimiento y evitar el alto costo de transmisión de video, el sistema distribuye el procesamiento en un modelo híbrido estructurado en capas. A continuación, se detalla el flujo de información para el caso de uso: **"Traducción de un gesto dinámico (palabra) a texto y audio"**.

- **Capa de Presentación y Preprocesamiento (Cliente Web):** El usuario accede a la plataforma desde el navegador de su dispositivo y enfoca a la persona. La aplicación web captura el video y ejecuta MediaPipe de forma local sobre el motor web para extraer únicamente los *landmarks* (una matriz tridimensional de 21 puntos clave espaciales de las manos), enviando solo este vector ligero en lugar de procesar imágenes completas (Google AI Edge, 2026).
- **Capa de Lógica de Negocio e Inferencia (Servidor Backend):** El controlador recibe el JSON y delega la carga al servicio de Inteligencia Artificial. En esta capa no existe entrenamiento; el servidor opera exclusivamente en fase de inferencia cargando en memoria modelos previamente entrenados de forma offline. Dependiendo del modo, el servicio rutea los datos:
  - *Modo Deletreo:* Pasa el vector estático a un clasificador SVM, ideal para clasificar coordenadas hiperespaciales fijas (Scikit-learn developers, 2026).
  - *Modo Conversación:* Pasa la secuencia temporal de vectores a una red LSTM, la cual está matemáticamente diseñada para procesar series temporales y mantener la memoria del movimiento (Keras Team, 2026).
- **Capa de Datos:** El texto predicho se almacena asíncronamente en una base de datos relacional utilizando PostgreSQL, reconocido por su robustez e integridad transaccional (PostgreSQL Global Development Group, 2026), registrando *id_usuario*, *texto_generado* y *timestamp* para alimentar el historial de la aplicación, garantizando la persistencia de la sesión.
- **Retorno al Cliente:** El servidor responde con un código HTTP 200 (Mozilla, 2026a) junto con el texto clasificado. La capa de presentación móvil recibe la respuesta, la despliega en pantalla e invoca la interfaz de síntesis de voz (API Text-to-Speech) del sistema para verbalizar el resultado de forma accesible (Mozilla, 2026b).

FIGURA 2 — DIAGRAMA DE ARQUITECTURA

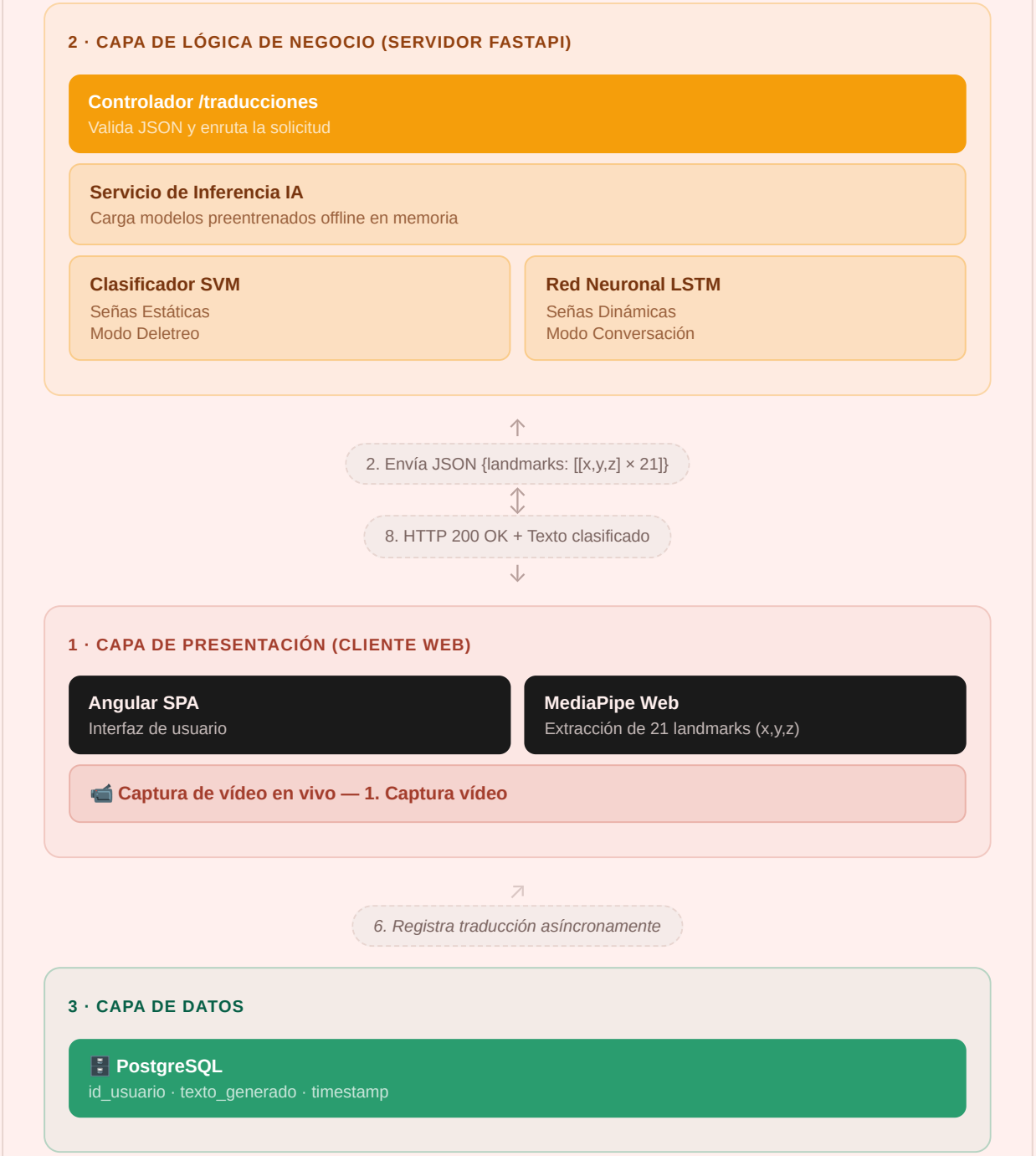


Figura 2 Diagrama de arquitectura

Selección de Tecnologías y Decisiones de Arquitectura

Para soportar el núcleo lógico y predictivo en el servidor (Backend), se evaluaron dos alternativas contrastando tres criterios:

1. Integración nativa con modelos preentrenados de IA
2. Curva de desarrollo
3. Capacidad de procesamiento concurrente

CANDIDATO A

Java — Spring Boot

Destaca por su estricto control de arquitectura empresarial y un manejo nativo de concurrencia multihilo superior, permitiendo aislar peticiones en hilos independientes, ideal para el procesamiento masivo transaccional (Spring, 2026).

CANDIDATO B

Python — FastAPI

Ofrece una velocidad de desarrollo alta y un rendimiento excelente (Tiangolo, 2026), poseyendo además el ecosistema nativo más robusto y compatible para cargar, interpretar y servir modelos matemáticos exportados desde Scikit-learn o Keras.

✓ Seleccionado

✓ **Decisión: Python con FastAPI para el desarrollo del servidor backend.**

Desventaja asumida: Al elegir Python, asumimos el costo de la limitación impuesta por el *Global Interpreter Lock* (GIL). El GIL es un mecanismo de bloqueo del intérprete que impide que múltiples hilos nativos ejecuten código simultáneamente (Python Software Foundation, 2026), lo que implica una eficiencia menor al paralelizar hilos frente a peticiones altamente concurrentes en comparación con Java. Aceptamos esta limitación técnica para priorizar la interoperabilidad fluida con nuestras herramientas de inferencia de aprendizaje automático y visión por computadora.

Para el cliente, se optará por el desarrollo de una Aplicación Web de Página Única (SPA) responsiva utilizando **Angular**, priorizando la accesibilidad universal a través del navegador del dispositivo sin requerir descargas desde tiendas de aplicaciones. El acceso a la cámara y el procesamiento de MediaPipe se ejecutarán directamente sobre el motor del navegador web mediante WebAssembly.

Desventaja asumida: Al prescindir de un empaquetado nativo (iOS/Android), asumimos una dependencia estricta de las políticas de seguridad de los navegadores web, lo que exige solicitar permisos de cámara en cada inicio de sesión si no se instala como aplicación progresiva (PWA), además de tener un acceso más limitado a los recursos de aceleración gráfica profunda del hardware frente a un desarrollo nativo.

SECCIÓN 3

Coste Monetario del Proyecto

Valorización Estimada — escenario de desarrollo de 2 semanas

Para determinar la viabilidad financiera de la propuesta, se han valorizado los costos de implementación del sistema asumiendo un escenario de desarrollo de 2 semanas. Las cifras presentadas a continuación son estimaciones referenciales en Pesos Chilenos (CLP).

COMPONENTE 1

Desarrollo — Horas-Hombre

$480.000 CLP

3 desarrolladores × 10 h/semana × 2 semanas = **60 horas-hombre** a $8.000 CLP/h. Cubre diseño de arquitectura, programación de la SPA en Angular y construcción de la lógica de negocio.

COMPONENTE 2

Infraestructura — Fase Prototipo

$0 CLP

El backend y el entorno de despliegue se encuentran en fase de definición. Todas las pruebas e inferencia se ejecutarán en entornos locales (localhost), sin costos operativos de servicios en la nube ni servidores externos.

COMPONENTE 3

Adquisición y Curación de Datos — Dataset LSCh

$480.000 CLP

Sin contratación de intérpretes. Se realizará minería de datos a partir de recursos abiertos de LSCh en internet. El proceso de extracción, limpieza y etiquetado de vectores (landmarks) consumirá aproximadamente **60 horas de trabajo técnico conjunto** a $8.000 CLP/h.

COMPONENTE 4

Licencias de Software y Herramientas

$0 CLP

Ecosistema 100 % Open Source: MediaPipe, Scikit-learn, Keras y PostgreSQL. Los entornos de desarrollo (IDEs JetBrains) están cubiertos bajo **licencias educativas**, eliminando gastos de software comercial.

COSTO TOTAL ESTIMADO

Valorización de la Propuesta

$960.000 CLP

SECCIÓN 3

Conclusión

Durante la fase de diseño de esta propuesta, se logró estructurar y validar técnicamente una arquitectura de software híbrida capaz de resolver el desafío de la traducción de la Lengua de Señas Chilena en tiempo real. En lugar de plantear una idea abstracta, se definió un flujo de trabajo asimétrico que viabiliza el sistema en un entorno web: delegar la extracción espacial de puntos clave al navegador del cliente para evitar el colapso de la red por transmisión de video, y centralizar la inferencia en el servidor mediante modelos de *Machine Learning* específicos para datos estáticos y secuenciales. Asimismo, se logró fundamentar cada decisión tecnológica basada en el costo-beneficio, dejando listo el contrato de la API y el esquema de persistencia para el inicio del desarrollo.

En cuanto a los alcances que quedan pendientes, el mayor desafío técnico por validar en iteraciones futuras es la optimización del modelo predictivo para mantener su precisión en escenarios con baja iluminación o fondos complejos, así como el proceso de recolección, etiquetado y curación del *dataset* específico de señas chilenas, un paso indispensable para ejecutar el entrenamiento *offline* de las redes neuronales antes de levantar el servidor a producción.

Finalmente, el principal aprendizaje de esta etapa radicó en la asimilación del rol de la arquitectura de software: tomar decisiones asumiendo compromisos. El equipo comprendió en la práctica que optar por Python (FastAPI) implicaba aceptar restricciones de procesamiento paralelo multihilo debido a su intérprete nativo (GIL), pero que este era un "costo arquitectónico" necesario para integrar sin fricciones el ecosistema de inteligencia artificial que da vida al producto. Se aprendió a no forzar a un solo modelo a resolver todo, dividiendo el problema matemático en algoritmos de clasificación (SVM) para posturas fijas y redes recurrentes (LSTM) para la memoria de los movimientos temporales.

REFERENCIAS

Bibliografía

Google AI Edge. (2026). *Hand landmarks detection guide*. Google for Developers. https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker

Instituto Nacional de Estadísticas. (2026, 11 febrero). Censo 2024. https://censo2024.ine.gob.cl/resultados/

Ionic. (2026). *Core Concepts: Cross Platform*. Ionic Framework Documentation. https://ionicframework.com/docs/core-concepts/cross-platform

Keras Team. (2026). *Timeseries forecasting*. Keras Documentation. https://keras.io/examples/timeseries/

Mozilla. (2026a). *200 OK - HTTP*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/HTTP/Status/200

Mozilla. (2026b). *SpeechSynthesis - Web APIs*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/SpeechSynthesis

PostgreSQL Global Development Group. (2026). *About PostgreSQL*. PostgreSQL Documentation. https://www.postgresql.org/about/

Python Software Foundation. (2026). *Initialization, Finalization, and Threads: The Global Interpreter Lock*. Python 3.14 Documentation. https://docs.python.org/3/c-api/threads.html

Scikit-learn developers. (2026). *Support Vector Machines*. Scikit-learn Machine Learning in Python. https://scikit-learn.org/stable/modules/svm.html

Spring. (2026). *Web on Servlet Stack*. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/web/webmvc.html

Tiangolo. (2026). *FastAPI*. FastAPI Documentation. https://fastapi.tiangolo.com/

MATERIAL DE RESPALDO

Anexos

ANEXO A

Declaración de uso de Inteligencia Artificial

En cumplimiento con las normativas de integridad académica del curso, el equipo declara el uso de asistentes de inteligencia artificial como apoyo durante la elaboración de esta propuesta, detallando su alcance a continuación:

- **Herramienta utilizada:** Gemini (Google).
- **Partes del trabajo donde se aplicó:** Apoyo en la estructuración de la arquitectura de información, redacción formal de la justificación técnica de la arquitectura cliente-servidor, comparativa de frameworks y limitantes del lenguaje, y organización del texto en la Introducción y Conclusión.
- **Método de verificación:** Los resultados generados por el modelo fueron revisados exhaustivamente por los integrantes del equipo. Se verificó que la arquitectura propuesta coincidiera con el diseño concebido por el grupo, se validaron las afirmaciones técnicas (como el impacto del GIL en Python y la integración de MediaPipe) contrastándolas con la documentación oficial citada en la bibliografía, y se adaptó la redacción final para asegurar que reflejara fielmente el alcance real del proyecto.